

Plany zapytań

SQL Server

Tryby przetwarzania

- Silnik bazy danych SQL Server przetwarza zapytania w różnych architekturach przechowywania danych, takich jak tabele lokalne, tabele partycjonowane i tabele rozproszone na wielu serwerach.
- **Tryby przetwarzania**
 - Row mode execution
 - to metoda przetwarzania zapytań używana z tradycyjnymi tabelami RDBMS, w których dane są przechowywane w formacie wierszowym.
 - Gdy zapytanie jest wykonywane i uzyskuje dostęp do danych w tabelach przechowywania wierszy, operatory drzewa wykonania i operatory podrzędne odczytują każdy wymagany wiersz, we wszystkich kolumnach określonych w schemacie tabeli.
 - Z każdego odczytanego wiersza SQL Server pobiera następnie kolumny wymagane dla zestawu wyników, do których odwołuje się instrukcja SELECT, predykat JOIN lub predykat filtrowania
 - Wykonywanie w trybie wierszowym jest bardzo wydajne w scenariuszach OLTP, ale może być mniej wydajne podczas skanowania dużych ilości danych, na przykład w scenariuszach hurtowni danych.



Tryby przetwarzania

- **Tryby przetwarzania**

- Batch mode execution

- jest metodą przetwarzania zapytań używaną do przetwarzania wielu wierszy razem (stąd termin batch(wsad)).
 - Każda kolumna w partii jest przechowywana jako wektor w oddzielnym obszarze pamięci, więc przetwarzanie w trybie wsadowym jest oparte na wektorach.
 - wykorzystuje również algorytmy zoptymalizowane pod kątem wielordzeniowych procesorów i zwiększonej przepustowości pamięci, które można znaleźć na nowoczesnym sprzęcie
 - począwszy od SQL Server 2019 (15.x) i Azure SQL Database, wykonywanie w trybie wsadowym nie wymaga już indeksów magazynu kolumn
 - działa na skompresowanych danych, gdy jest to możliwe, i eliminuje operator wymiany(Parallelism) używany w trybie wierszowym. Rezultatem jest lepsza równoległość i wyższa wydajność.
 - Gdy zapytanie jest wykonywane w trybie wsadowym i uzyskuje dostęp do danych w indeksach magazynu kolumn, operatory drzewa wykonania i operatory podrzędne odczytują wiele wierszy razem w segmentach kolumn. SQL Server odczytuje tylko kolumny wymagane dla wyniku, do których odwołuje się instrukcja SELECT, predykat JOIN lub predykat filtru.

Przetwarzanie zapytań

- **Parsowanie**
 - Sprawdzanie składni
 - Tworzenie drzewa parsowania
- **Przypisywanie**
 - Rozwiązywanie nazw obiektów i kolumn
 - Tworzenie drzewa zalgabraizowanego
- **Optymalizacja**
 - Generowanie możliwych planów wykonania
 - Wybór planu (czasem tylko wystarczający, nie najlepszy)
 - Plan trywialny, plan pełny
 - 3 fazy optymalizacji(searcho, search1, search2)
- **Wykonanie zapytania**
 - Wykonanie zapytania odczytując strony danych z pamięci i/bądź plików

Parsowanie
(query parsing)

Przypisywanie
(query algebrizer)

Optymalizacja
(query optimizer)

Wykonanie
(query execution)

Przetwarzanie zapytań

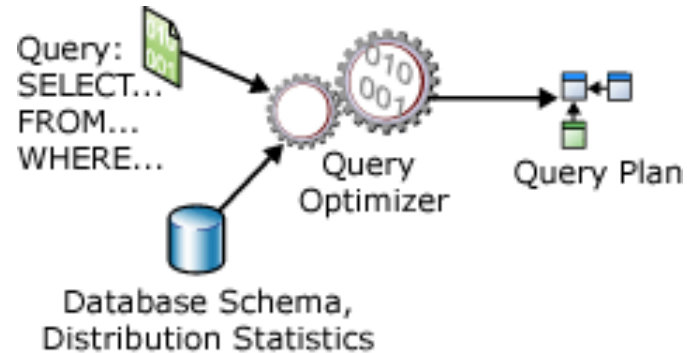
- Opis poprzedniego slajdu:
 - Parser skanuje instrukcję SELECT i dzieli ją na jednostki logiczne, takie jak słowa kluczowe, wyrażenia, operatory i identyfikatory.
 - Tworzone jest drzewo zapytań, czasami określane jako drzewo sekwencji, opisujące logiczne kroki potrzebne do przekształcenia danych źródłowych do formatu wymaganego przez zestaw wyników.
 - Optymalizator zapytań analizuje różne sposoby dostępu do tabel źródłowych. Następnie wybiera serię kroków, które najszybciej zwracają wyniki przy mniejszym zużyciu zasobów. Drzewo zapytań jest aktualizowane w celu zapisania dokładnej serii kroków. Ostateczna, zoptymalizowana wersja drzewa zapytań nazywana jest planem wykonania.
 - Silnik relacyjny rozpoczyna wykonywanie planu wykonania. Gdy kroki wymagające danych z tabel bazowych są przetwarzane, silnik relacyjny żąda, aby silnik pamięci masowej przekazał dane z zestawów wierszy żądanych z silnika relacyjnego.
 - Silnik relacyjny przetwarza dane zwrócone z silnika magazynu do formatu zdefiniowanego dla zestawu wyników i zwraca zestaw wyników do klienta.

Przetwarzanie zapytań - Search

- **Search 0**
 - Na etapie wykonywane są operacje analogiczne jak w przypadku planu trywialnego
- **Search 1**
 - optymalizator zapytań dokonuje zmian kolejności złączeń, wybierając je od najmniej kosztownych
 - Zmiany te mogą być odpytywane kilkakrotnie w celu korelacji wielu parametrów (ilość I/O, zużycie pamięci ram, zużycie procesora) mających wpływ na ostateczne wykonanie zapytania
- **Search 2**
 - Z kolei na tym etapie optymalizator zapytań wykorzystuje dane zebrane w poprzednim etapie (Search 1) i biorąc pod uwagę czas wykonania w korelacji z kosztami wybiera najbardziej optymalny plan wykonania zapytania

Przetwarzanie zapytania SQL

- Optymalizacja instrukcji SELECT



- Instrukcja SELECT definiuje tylko następujące elementy:
 - Format zestawu wyników. Jest on określony głównie w liście select. Jednak inne klauzule, takie jak ORDER BY i GROUP BY, również wpływają na ostateczną formę zestawu wyników.
 - Tabele zawierające dane źródłowe. Jest to określone w klauzuli FROM.
 - W jaki sposób tabele są logicznie powiązane dla celów instrukcji SELECT. Jest to zdefiniowane w specyfikacjach złączenia, które mogą pojawić się w klauzuli WHERE lub w klauzuli ON następującej po FROM.
 - Warunki, które muszą spełniać wiersze w tabelach źródłowych, aby zakwalifikować się do instrukcji SELECT. Są one określone w klauzulach WHERE i HAVING.

Przetwarzanie zapytania SQL

- **Plan wykonania zapytania jest definicją następujących elementów:**
 - Kolejność dostępu do tabel źródłowych
 - Zazwyczaj istnieje wiele sekwencji, w których serwer bazy danych może uzyskać dostęp do tabel bazowych w celu zbudowania zestawu wyników. Na przykład, jeśli instrukcja SELECT odwołuje się do trzech tabel, serwer bazy danych może najpierw uzyskać dostęp do tabeli A, użyć danych z tabeli A do wyodrębnienia pasujących wierszy z tabeli B, a następnie użyć danych z tabeli B do wyodrębnienia danych z tabeli C.
 - Metody używane do wyodrębniania danych z każdej tabeli.
 - istnieją różne metody dostępu do danych w każdej tabeli.
 - Jeśli wymaganych jest tylko kilka wierszy z określonymi wartościami klucza, serwer bazy danych może użyć indeksu.
 - Jeśli wymagane są wszystkie wiersze w tabeli, serwer bazy danych może zignorować indeksy i wykonać skanowanie tabeli.
 - Jeśli wymagane są wszystkie wiersze w tabeli, ale istnieje indeks, którego kluczowe kolumny znajdują się w ORDER BY, wykonanie skanowania indeksu zamiast skanowania tabeli może zaoszczędzić osobnego sortowania zestawu wyników.
 - Jeśli tabela jest bardzo mała, skanowanie tabeli może być najbardziej wydajną metodą dla prawie całego dostępu do tabeli.

Optymalizacja (1/5)

- Proces wyboru jednego planu wykonania z potencjalnie wielu możliwych planów nazywany jest optymalizacją.
- Optymalizator zapytań jest jednym z najważniejszych komponentów silnika bazy danych.
- Podczas gdy optymalizator zapytań wykorzystuje pewien narzut do analizy zapytania i wyboru planu, ten narzut jest zwykle kilkakrotnie oszczędzany, gdy optymalizator zapytań wybiera efektywny plan wykonania.
- Na przykład, dwie firmy budowlane mogą otrzymać identyczne plany domu. Jeśli jedna firma poświęci na początku kilka dni na zaplanowanie, w jaki sposób zbuduje dom, a druga firma rozpocznie budowę bez planowania, firma, która poświęci czas na zaplanowanie swojego projektu, prawdopodobnie ukończy go jako pierwsza.

Optymalizacja (2/5)

- Optymalizator zapytań SQL Server jest **optymizatorem opartym na kosztach**.
- Każdy możliwy plan wykonania ma powiązany koszt w postaci ilości wykorzystanych zasobów obliczeniowych.
- Optymalizator zapytań musi przeanalizować możliwe plany i wybrać ten o najniższym szacowanym koszcie.
- Niektóre złożone instrukcje SELECT mają tysiące możliwych planów wykonania. **W takich przypadkach Query Optimizer nie analizuje wszystkich możliwych kombinacji.**
- Zamiast tego używa złożonych algorytmów, aby znaleźć plan wykonania, który ma koszt rozsądnie zbliżony do minimalnego możliwego kosztu.

Optymalizacja (3/5)

- Optymalizator zapytań SQL Server **nie wybiera tylko planu wykonania o najniższym koszcie zasobów;**
- wybiera plan, który zwraca wyniki użytkownikowi przy rozsądnym koszcie zasobów i który zwraca wyniki najszybciej.
- Przykładowo, równoległe przetwarzanie zapytania zazwyczaj zużywa więcej zasobów niż przetwarzanie szeregowe, ale kończy zapytanie szybciej.
- Optymalizator zapytań SQL Server użyje równoległego planu wykonania, aby zwrócić wyniki, jeśli nie wpłynie to negatywnie na obciążenie serwera.

Optymalizacja (4/5)

- Optymalizator zapytań SQL Server opiera się na statystykach, gdy szacuje koszty zasobów różnych metod wyodrębniania informacji z tabeli lub indeksu.
- Statystyki są przechowywane dla kolumn i indeksów i zawierają informacje o gęstości danych bazowych.
 - Gęstość określa rozkład unikalnych wartości istniejących w danych lub średnią liczbę zduplikowanych wartości dla danej kolumny. Wraz ze spadkiem gęstości wzrasta selektywność wartości.
- Jest to wykorzystywane do wskazania selektywności wartości w określonym indeksie lub kolumnie.
- Na przykład, w tabeli reprezentującej samochody, wiele samochodów ma tego samego producenta, ale każdy samochód ma unikalny numer identyfikacyjny pojazdu (VIN). Indeks VIN jest bardziej selektywny niż indeks producenta, ponieważ VIN ma mniejszą gęstość niż producent.
- Jeśli statystyki indeksu nie są aktualne, optymalizator zapytań może nie dokonać najlepszego wyboru dla bieżącego stanu tabeli.

Optymalizacja (5/5)

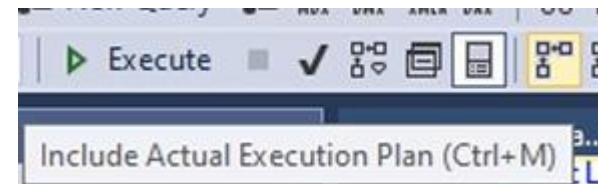
- Optymalizator zapytań SQL Server jest ważny, ponieważ umożliwia serwerowi bazy danych dynamiczne dostosowywanie się do zmieniających się warunków w bazie danych bez konieczności wprowadzania danych przez programistę lub administratora bazy danych.
- Dzięki temu programiści mogą skupić się na opisywaniu końcowego wyniku zapytania. Mogą ufać, że SQL Server Query Optimizer zbuduje wydajny plan wykonania dla stanu bazy danych za każdym razem, gdy instrukcja jest uruchamiana.

Rodzaje operatorów

- **Estimated Execution Plan**
 - Szacowany plan wykonania



- **Actual Execution Plan**
 - Właściwy plan zapytania



- Jeśli statystyki są nieaktualne, to plan szacowany może się różnić od właściwego

Właściwości operatorów

100%	Index Scan (NonClustered)
ame, p.C	Scan a nonclustered index, entirely or only a range.
	Physical Operation Index Scan
Scan (NonC	Logical Operation Index Scan
X_Person_Li	Actual Execution Mode Row
Cost: 2	Estimated Execution Mode Row
0.005s	Storage RowStore
19972 o	Number of Rows Read 19972
19972 (10	Actual Number of Rows 19972
	Actual Number of Batches 0
Hash Matr	Estimated I/O Cost 0,080162
(Inner Jo:	Estimated Operator Cost 0,102288 (2%)
Cost: 10	Estimated CPU Cost 0,0221262
0.206s	Estimated Subtree Cost 0,102288
121317 o	Number of Executions 1
70624 (17	Estimated Number of Executions 1
	Estimated Number of Rows 19972
	Estimated Number of Rows to be Read 19972
	Estimated Row Size 117 B
	Actual Rebinds 0
	Actual Rewinds 0
	Ordered False
	Node ID 1
	Object
	[AdventureWorks2017].[Person].[Person].
	[IX_Person_LastName_FirstName_MiddleName] [pe]
	Output List
	[AdventureWorks2017].[Person].[Person].BusinessEntityID;
	[AdventureWorks2017].[Person].[Person].FirstName;
	[AdventureWorks2017].[Person].[Person].LastName

Właściwości planów

Properties	
SELECT	
Misc	
Cached plan size	176 KB
CardinalityEstimationModelVersion	140
CompileCPU	30
CompileMemory	1528
CompileTime	30
Degree of Parallelism	0
Estimated Number of Rows	68125,9
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	6,22716
Memory Grant	28328
MemoryGrantInfo	
NonParallelPlanReason	CouldNotGenerateValidParallelPlan
Optimization Level	FULL
OptimizerHardwareDependentProperties	
OptimizerStatsUsage	
QueryHash	0xD8BC2FE162EF8AE9
QueryPlanHash	0xFE49EE67D35A74DB
QueryTimeStats	
RetrievedFromCache	true
SecurityPolicyApplied	False
Set Options	
ANSI_NULLS	True; ANSI_PADDING
Statement	SELECT soh.SalesOrderID, soh.OrderDate,
WaitStats	

Query 1: Query cost (relative to the
SELECT soh.SalesOrderID, soh.OrderDate,

Hash Match (Inner Join)	
SELECT	
Cost: 0	
Cached plan size	176 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	0
Estimated Subtree Cost	6,22716
Memory Grant	28328
Estimated Number of Rows	68125,9
Statement	
SELECT	
soh.SalesOrderID, soh.OrderDate,	
p.Name, p.Color, sod.OrderQty,	
pe.FirstName, pe.LastName,	
c.CustomerID	
FROM Sales.SalesOrderHeader AS soh	
JOIN Sales.SalesOrderDetail AS sod ON	
sod.SalesOrderID = soh.SalesOrderID	
JOIN Production.Product AS p ON	
p.ProductID = sod.ProductID	
JOIN Production.ProductSubcategory AS	
ps ON ps.ProductSubcategoryID =	
p.ProductSubcategoryID	
JOIN Production.ProductCategory AS pc	
ON pc.ProductCategoryID =	
ps.ProductCategoryID	

Optimization Level

Properties	
SELECT	
Misc	
Cached plan size	40 KB
CardinalityEstimationMod	150
CompileCPU	1
CompileMemory	272
CompileTime	1
DatabaseContextSettingslc	1
Degree of Parallelism	1
Estimated Number of Row	1
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	0,0032831
MemoryGrantInfo	
Optimization Level	TRIVIAL
OptimizerHardwareDepen	
Parameter List	
ParentObjectId	0
QueryHash	0xE3B18E2D9297521B
QueryPlanHash	0xBE8C1CFFE602F0B6
QueryTimeStats	

Optymalizator zapytań programu SQL Server jest optymalizatorem opartym na kosztach i generuje różnych kandydatów na plany zapytań, a następnie decyduje o najbardziej kosztownym planie.

Scenariusz ulegnie jednak zmianie, jeśli przesłane zapytanie będzie bardzo proste i dostępny będzie tylko jeden możliwy plan.

W tej sytuacji optymalizator ignoruje znalezienie planu optymalnego, ponieważ wynik tego działania będzie tylko stratą czasu.

Plany tego typu nazywamy planami TRIVIALNYmi, które powstają poprzez pominięcie procesów optymalizacji zapytań dla prostych zapytań.

Koszt operatora

- Cost to abstrakcyjna wartość
- Dziś to miara bez jednostek
 - To nie czas
 - To nie CPU
- Koszt to ZAWSZE szacunek (estimate)
- W 1998 roku na komputerze jednego z inżynierów Microsoft powstał ten operator i wtedy oznaczał sekundy (SQL Server 7)
- Dziś operacja trwająca 10 sekund na tamtym komputerze wykona się dużo szybciej

Estimated Operator Cost

0,57557 (35%)

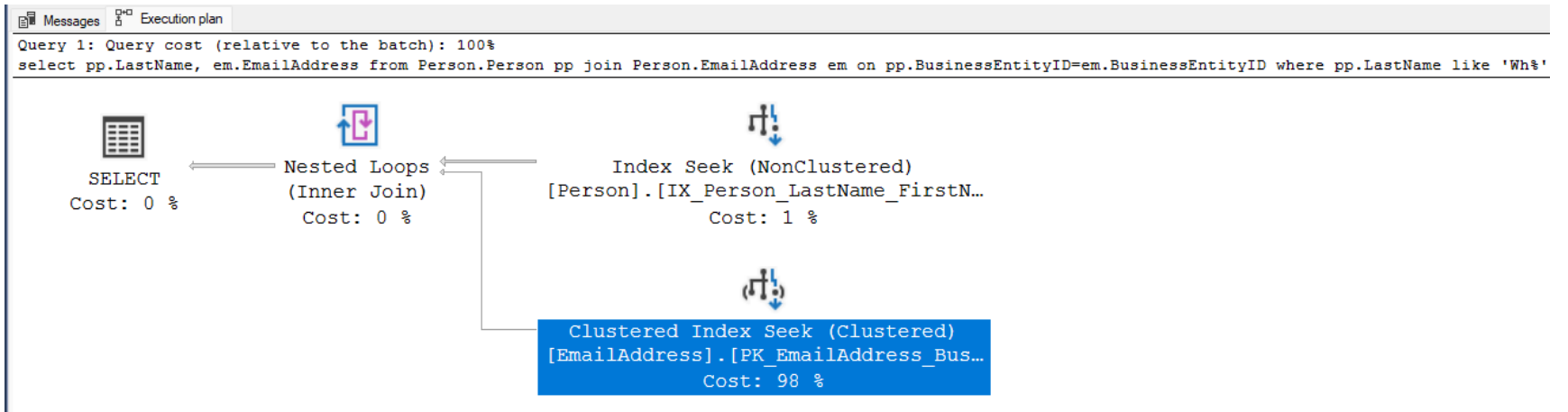
Koszt operatora

- Estimated Operator Cost – szacowany koszt operatora, również procent całej operacji
- **Estimated I/O cost - koszt operacji dyskowych**
- Estimated CPU cost – koszt procesora
- Estimated row size – szacowana liczba wierszy, na jej podstawie określany jest rozmiar pamięci
- Number of Executions – ile razy operator był użyty
- **Actual and Estimated number of rows – liczba wierszy**
- Actual and Estimated execution mode – row/ batch

Hash Match	
Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.	
Physical Operation	Hash Match
Logical Operation	Aggregate
Actual Execution Mode	Row
Estimated Execution Mode	Row
Actual Number of Rows	266
Actual Number of Batches	0
Estimated Operator Cost	0,57557 (35%)
Estimated I/O Cost	0
Estimated CPU Cost	0,575573
Estimated Subtree Cost	1,62934
Number of Executions	1
Estimated Number of Executions	1
Estimated Number of Rows	266
Estimated Row Size	11 B
Actual Rebinds	0
Actual Rewinds	0
Node ID	0

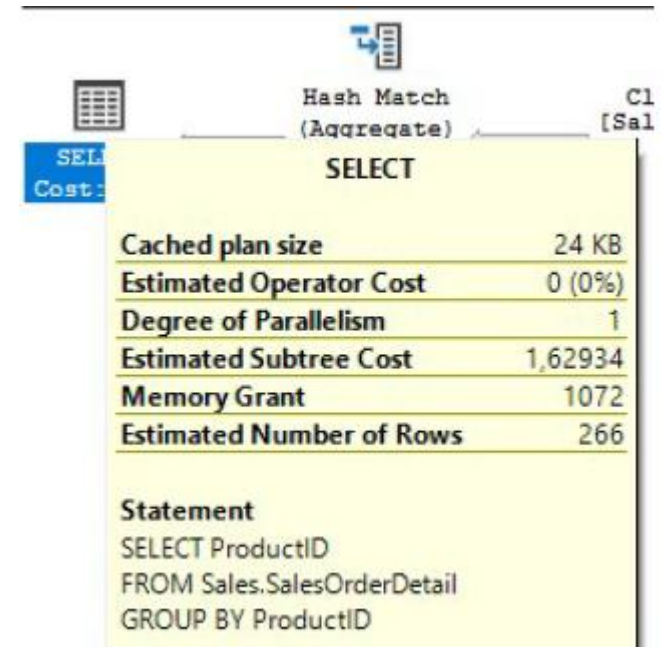
Jak czytać plan?

- Można go czytać z obu stron
 - Od lewej do prawej
 - Od prawej do lewej
- FLOW Control (od lewej -> do prawej)
- DATA Control (od prawej -> do lewej)
- Dane o planie otrzymasz z:
 - SQL Managment Studio – graficznie
 - SQL Managment Studio – XML
 - SQL Sentry Plan Explorer



Właściwości planu

- **Cached Plan Size** – rozmiar pamięci potrzebnej na zapisanie planu
- **Degree of Parallelism** – stopień zrównoleglenia
- **Estimated Subtree Cost** – szacowany koszt planu zapytania (suma kosztów poszczególnych operatorów)
- **Memory Grant** – zarezerwowana pamięć na potrzeby wykonywania zapytania (w szczególności Hash Match/ Sort)
- **Estimated Number of Rows** – szacowana liczba wierszy, na podstawie statystyk



The screenshot shows a SQL Server query plan for a Hash Match (Aggregate) operator. The operator's icon is a grid with a blue arrow. The statistics window is open, displaying various performance metrics for the selected operator. The background of the statistics window is yellow.

	C1 [Sal
SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	1,62934
Memory Grant	1072
Estimated Number of Rows	266
Statement	
SELECT ProductID	
FROM Sales.SalesOrderDetail	
GROUP BY ProductID	

Operatory – dostęp do danych

- Clustered Index Seek (1)
- Index Seek (2)
- Clustered Index Scan (3)
- Index Scan (Nonclustered) (4)
- Key Lookup – doczytujemy z indeksu klastrowego (5)
- RID Lookup – doczytujemy ze sterty (6)
- Table scan – tylko na stercie (7)

Kluczowe elementy w planie zapytania

- Sprawdź pierwszy operator
- Znajdź najbardziej wymagające operatory
- Ostrzeżenia
- Nieznane operatory
- Grube strzałki pomiędzy operatorami(przesłanie sporej ilości danych)
- Operator skanowania (nie zawsze jest problemem)
- Estymowany vs aktualny plan zapytania

Operatory – Łączenie danych

- **Nested Loops Join**
 - Wsad zewnętrzny wykonywany tylko raz, a wewnętrzny w pętli
- **Merge Join**
 - Oba operatory odczytujące dane są wykonywane tylko raz, ale muszą być posortowane zgodnie z kluczem złączenia
- **Hash Match**
 - Używany dla dużych, nieposortowanych tabel
 - Dla mniejszej tabeli tworzona jest tabela haszująca w pamięci

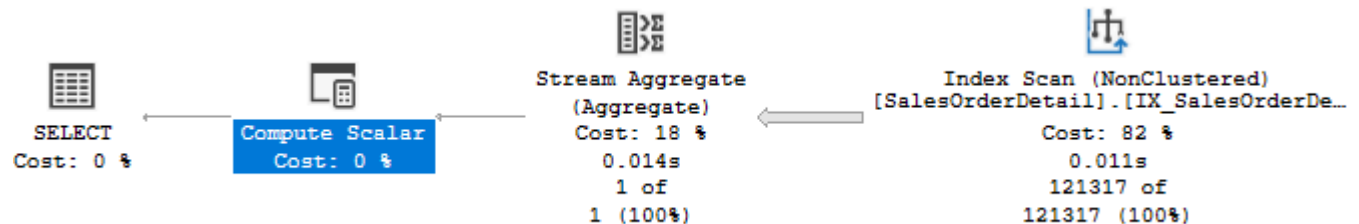
Operatory – Grupowanie

- **Stream Aggregate**
 - Dane muszą być posortowane po kolumnie z klauzuli GROUP BY
- **Hash Aggregate**
 - Tworzy tabelę hash-ującą żeby zdefiniować grupy

Operatory – Compute Scalar

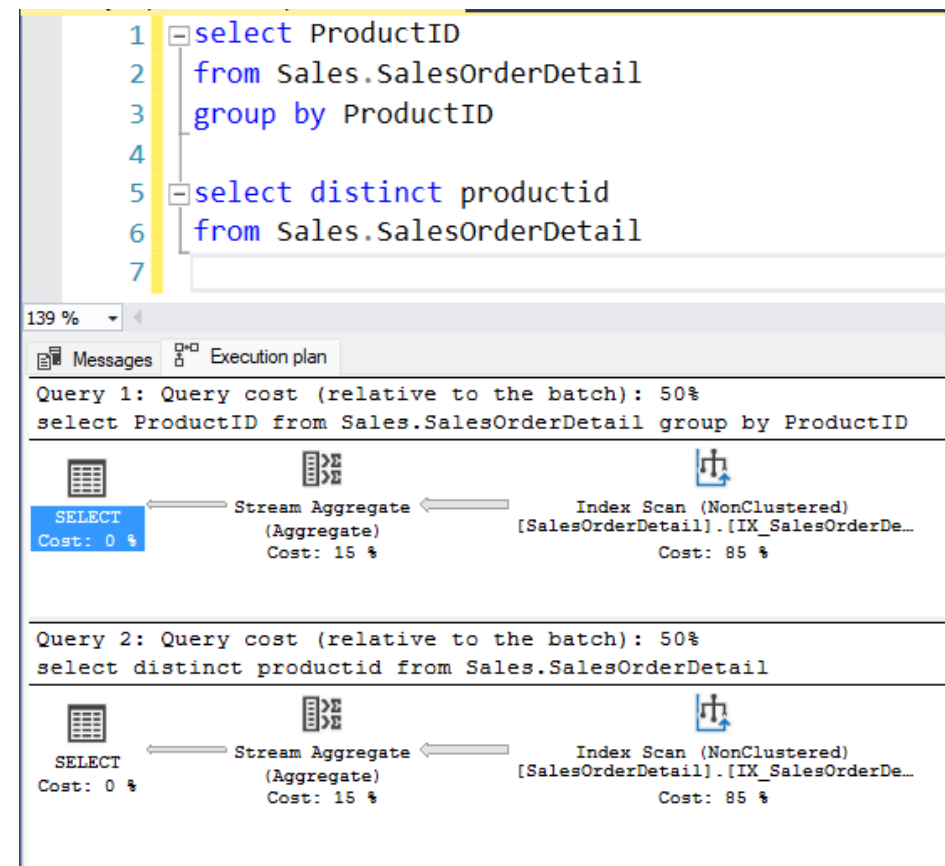
- **Compute Scalar**

- Służy do obliczania nowych wartości na podstawie innych kolumn w tym samym wierszu. Te nowe wartości są następnie dodawane jako dodatkowe kolumny w wierszach wyjściowych
- Koszt w zasadzie nie może istnieć (zwłaszcza dla funkcji zdefiniowanych przez użytkownika)
- W rezultacie szacowany koszt może być naprawdę niski i totalnie mylący dla optymalizatora



Operatory – usuwanie duplikatów

- Stream Aggregate
- Distinct Sort
- Hash Aggregate



Operatory – Plany równoległe – operatory wymiany

- **Distribute Streams**
 - Dzieli pojedynczy strumień na wiele wątków
- **Gather Streams**
 - Przyjmuje kilka wątków, łączy je i tworzy pojedynczy wątek
- **Repartition Streams**
 - Pobiera wiele wątków i produkuje wiele wątków (równoległe łączenie tabel)

Dlaczego?

- Dlaczego zapytania NIE wykonują się równolegle?
 - Ustawienie **MAXDOP** (na poziomie zapytania, w konfiguracji bazy danych bądź instancji)
 - Koszt zapytania poniżej **cost threshold for parallelism**
 - Użycie w zapytaniu kursorów dynamicznych, funkcji rankingu, funkcji skalarnych użytkownika, tabel memory-optimized
 - Edycja Express

Memory Grant	1040
⊕ MemoryGrantInfo	
NonParallelPlanReason	CouldNotGenerateValidParallelPlan
Optimization Level	FULL
⊕ OptimizerHardwareDep	

Cache planów (Query cache plan)

- **sys.dm_exec_cached_plans** – cache planów dla całej instancji
 - rozmiarem planów w bajtach
 - liczba użyć
 - identyfikator pozwalający odczytać treść instrukcji/ plan zapytania
- **sys.dm_exec_query_stats** – statystyki planów
 - creation_time
 - last_execution_time
 - total/last/min/max_logical_reads
 - total/last/min/max_rows
- **Czyszczenie planu zapytań**
 - Skasowanie wybranego planu - DBCC FREEPROCCACHE (plan_handle)
 - Skasowanie wszystkich planów - DBCC FREEPROCCACHE
 - Skasowanie planów dla bieżącej bazy ALTER DATABASE SCOPED CONFIGURATION, CLEAR PROCEDURE_CACHE

Parameter Sniffing

- W przeciwieństwie do zapytań ad-hoc, procedury składowane korzystają tylko z jednego planu wykonania, bez względu na inną kombinację parametrów.
- SQL Server tworzy i zapisuje plan wykonania na podstawie PIERWSZEGO zestawu parametrów.
- Parameter Sniffing – rozwiązania
 - `sp_recompile 'NAZWA_PROCEDURY'`
 - `CREATE PROC .... WITH RECOMPILE`
 - `OPTION (RECOMPILE)`
 - `OPTION (OPTIMIZE FOR UNKNOWN)`
 - `OPTIMIZE FOR X`
 - Zagnieżdżona procedura